

CS 636 Semester 2024–2025-II: Assignment 2

Name: **Vinayak Devvrat**

Roll Number: **241110079**

April 12, 2025

NOTE:

- All the codes for this assignment have been compiled and executed on KD first floor workstation with ID: **CSEWS34**.
- At the time of submitting this report, the KD systems did not have support for Google Test Framework. Therefore, I ran the unit tests on my system. Screenshots can be found in the `screenshots/unit-tests` directory.

Problem 1

Parts (i) to (iii)

Please refer to the attached code.

Part (iv)

Following are the time values recorded for different operations of my PThread implementation.

Table 1: Insert

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
8	3.5	37.5	349.5
16	3.5	34	340.5

Table 2: Delete

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
8	144	1932.5	20615.5
16	117	1752.5	18931.5

Table 3: Lookup

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
8	376.5	4007.5	42755.5
16	309	3596	38663

Part (v)

Please refer to the attached code.

Part (vi)

The following tables present the throughput comparisons of the PThread implementation against the TBB implementation.

Table 4: Insert

Number of threads	1e5 (in ops/sec)		1e6 (in ops/sec)		1e7 (in ops/sec)	
	Pthread	TBB	Pthread	TBB	Pthread	TBB
8	1.14e7	8e7	1.07e7	1.6e8	1.44e7	1.77e8
16	1.14e7	8e7	1.17e7	8e7	1.17e7	2.05e8

Table 5: Delete

Number of threads	1e5 (in ops/sec)		1e6 (in ops/sec)		1e7 (in ops/sec)	
	Pthread	TBB	Pthread	TBB	Pthread	TBB
8	138889	∞	103493	2e8	97014.4	2.85e8
16	170940	∞	114123	∞	105644	3.33e8

Table 6: Lookup

Number of threads	1e5 (in ops/sec)		1e6 (in ops/sec)		1e7 (in ops/sec)	
	Pthread	TBB	Pthread	TBB	Pthread	TBB
8	53120.9	∞	49906.4	1e8	46777.6	9.09e7
16	64724.9	∞	55617.4	1e8	51729	1.05e8

Execution Command : `./problem1.out -ops=<num_operations> -thr=<num_threads> -tbb=[0|1]`

Problem 2

Parts (i) to (iii)

Please refer to the attached code.

Parts (iv) and (vi)

The following table summarises the time taken to complete n enqueue and dequeue operations (randomly determined) and a scalability analysis with several threads counts.

Table 1: My Implementation

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
1	139	1449	14090
2	75	734	7304
4	38	374	3698
8	25	257	2380
16	21	192	1820

Part (v)

The following tables summarise the comparison and scalability of various concurrent queue implementations.

Table 2: Intel TBB

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
1	139	1424	13861
2	72	711	7076
4	36	358	3575
8	26	251	2320
16	44	486	3767

Table 3: boost::lockfree::queue

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
1	172	1736	20723
2	104	1021	10326
4	81	666	6750
8	57	565	5401
16	51	500	4894

Table 4: moodycamel::ConcurrentQueue

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
1	142	1382	13713
2	72	717	7091
4	36	366	3686
8	25	251	2350
16	20	198	1807

- Based on the observations, it can be concluded that my implementation is comparable to the other three implementations for any given workload.
- Out of the three other implementations, moodycamel::ConcurrentQueue is the most scalable.

Execution Command : `./problem2.out -ops=<num_operations> -thr=<num_threads> [ -tbb | -moody | -boost ]`

If no implementation flag is specified, it would default to my implementation.

Problem 3

Parts (i) to (iii)

Please refer to the attached code.

Parts (iv) and (vi)

Table 1: Reported Time and Scalability Analysis

Number of threads	1e5 (in ms)	1e6 (in ms)	1e7 (in ms)
1	142	1414	14066
2	75	740	7358
4	39	376	3800
8	28	291	2992
16	22	198	1934

Execution Command : `./problem3.out -ops=<num_operations> -thr=<num_threads>`